

SIMATS ENGINEERING

CO3- AT2 - Design Task - Medium

COURSE NAME: Advance Wireless Communication Systems /

5G / 6G Communication Systems

COURSE CODE: ECA56

COURSE OUTCOME COVERED

CO: 3 - Design spectrum access strategies, waveform generation methods, and multiple access techniques for efficient wireless transmission systems. (BL6)

Assessment Tool Weightage: 35%

Short description about assessment: Students design a system / model based on given requirements to assess creativity and application skills.

Dr.

QUESTIONS: 1 (1 X 100 = 100)

Total Marks: 100 Duration: 3hrs

1. Design a system architecture for Multi-access Edge Computing to reduce latency in smart traffic applications. (BL6)
2. Develop a cloud-edge communication model for wireless healthcare monitoring. (BL6)
3. Design a functional architecture for real-time edge data processing in industrial automation. (BL6)
4. Propose a cloud-based wireless framework for smart surveillance systems. (BL6)
5. Design an edge AI workflow for intelligent wireless data filtering. (BL6)
6. Develop a task offloading strategy between edge and cloud in 5G applications. (BL6)
7. Design a low-latency communication architecture for edge-assisted smart city services. (BL6)
8. Propose a scalable edge-cloud framework for wireless sensor networks. (BL6)
9. Design a Cloud Radio Access Network model for dense urban deployment. (BL6)
10. Develop a functional split model between centralized and distributed radio units. (BL6)
11. Design a virtualized wireless access framework for next-generation communication. (BL6)
12. Propose a data flow model for cloud-based radio resource allocation. (BL6)
13. Design a centralized traffic control model using Cloud RAN principles. (BL6)
14. Develop a network management architecture for cloud-assisted wireless systems. (BL6)
15. Design a wireless service framework integrating virtual network functions. (BL6)
16. Propose an optimized fronthaul communication model. (BL6)
17. Design a satellite-assisted communication model using Low Earth Orbit satellite for remote wireless coverage. (BL6)
18. Develop a comparative architecture for LEO, MEO, and GEO backhaul systems. (BL6)

19. Design a spectrum sharing model between LTE and 5G users. (BL6)
20. Propose a frequency allocation strategy for mixed wireless services. (BL6)
21. Design a dynamic spectrum access framework for urban wireless systems. (BL6)
22. Develop a communication architecture combining terrestrial and satellite networks. (BL6)
23. Design a spectrum efficiency improvement model for dense communication zones. (BL6)
24. Propose a channel reuse plan for multi-user wireless communication. (BL6)
25. Design an NB-IoT architecture for smart utility monitoring. (BL6)
26. Develop an LTE-M communication workflow for wearable monitoring systems. (BL6)
27. Design a low-power communication model for battery-operated IoT devices. (BL6)
28. Propose a device communication plan for long-life sensor networks. (BL6)
29. Design a 5G-enabled IoT framework for campus automation. (BL6)
30. Develop an energy-efficient wireless communication strategy for sensor nodes. (BL6)
31. Design an IoT communication flow for periodic environmental sensing. (BL6)
32. Propose a scalable wireless architecture for smart connected devices. (BL6)
33. Design a smart city communication framework for traffic monitoring and surveillance. (BL6)
34. Develop an energy monitoring communication model for public infrastructure. (BL6)
35. Design a smart factory information flow for predictive maintenance. (BL6)
36. Propose a robotic automation communication framework for industrial control. (BL6)
37. Design a remote patient monitoring communication architecture. (BL6)
38. Develop a wireless communication workflow for AI-assisted healthcare diagnosis. (BL6)
39. Design a smart irrigation communication model for crop monitoring. (BL6)
40. Propose a livestock tracking communication architecture using wireless sensors. (BL6)

Code of Conduct:

I B. Yagneswar Reddy (192312285) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B.Yagneswar

RUBRICS FOR CALCULATION:

Design Task					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Design	20%	Demonstrates deep understanding of design objectives, constraints, and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Lacks understanding of the design context
Evaluation of Design	25%	Provides in-depth critique identifying strengths, weaknesses, and design gaps	Identifies key issues with reasonable analysis	Limited critique; mostly descriptive feedback	No meaningful critique provided
Justification	20%	Supports critique with strong reasoning, evidence, and relevant principles	Provides justification with some supporting evidence	Weak or unclear justification	No justification for feedback
Suggestions for Improvement	20%	Offers innovative, practical, and well-justified improvement suggestions	Provides useful suggestions with minor gaps	Suggestions are limited or partially relevant	No constructive suggestions provided
Communication	15%	Communicates feedback clearly, professionally, and engages actively in discussion	Communicates well with minor issues	Communication lacks clarity or engagement	Poor communication and minimal participation

Task Offloading Strategy between Edge and Cloud in 5G Applications

1. Understanding of Design — Objectives, Constraints and Context

1.1 Problem Context

Modern 5G applications — augmented/virtual reality, real-time video analytics, connected-vehicle perception, and industrial vision — generate compute-intensive tasks that a resource-constrained user device (UE) often cannot process within the application's latency deadline or energy budget. 5G introduces Multi-access Edge Computing (MEC) hosts co-located at or near the base station (gNB), giving the network operator a third option — beyond pure local execution and distant public cloud — for where a task's computation actually happens. The design problem is to decide, for every task, whether it should run locally on the UE, be offloaded to the nearby edge, or be forwarded further to the centralised cloud.

1.2 Design Objectives

- **Latency compliance:** meet each application's latency deadline regardless of where the task executes.
- **Energy efficiency:** minimise UE battery drain by favouring execution locations that reduce total energy consumption.
- **Resource efficiency:** continuously balance load between edge and cloud so neither tier is persistently overloaded.
- **Scalability:** support many concurrent UEs and heterogeneous task types within one unified decision framework.
- **Adaptivity:** re-evaluate the offloading decision as channel quality, device load and server load change over time.

1.3 Design Constraints

- **Radio resource:** the uplink data rate available for offloading is limited by the allocated 5G NR bandwidth and instantaneous channel SINR.
- **Finite edge capacity:** edge servers have far less aggregate compute and storage capacity than a centralised cloud data-centre.
- **Backhaul latency:** the cloud offers abundant compute but incurs additional backhaul/core-network round-trip delay the edge does not.
- **Device energy budget:** the UE's battery state constrains how aggressively it can rely on local computation.
- **Decision overhead:** the decision engine itself must run with negligible overhead so it does not become the new bottleneck.

2. Proposed Task Offloading Architecture and Workflow

The proposed design organises the system into five functional layers with an explicit offloading-decision workflow, as shown in Figure 1.

Task Offloading Strategy between Edge and Cloud in 5G Applications

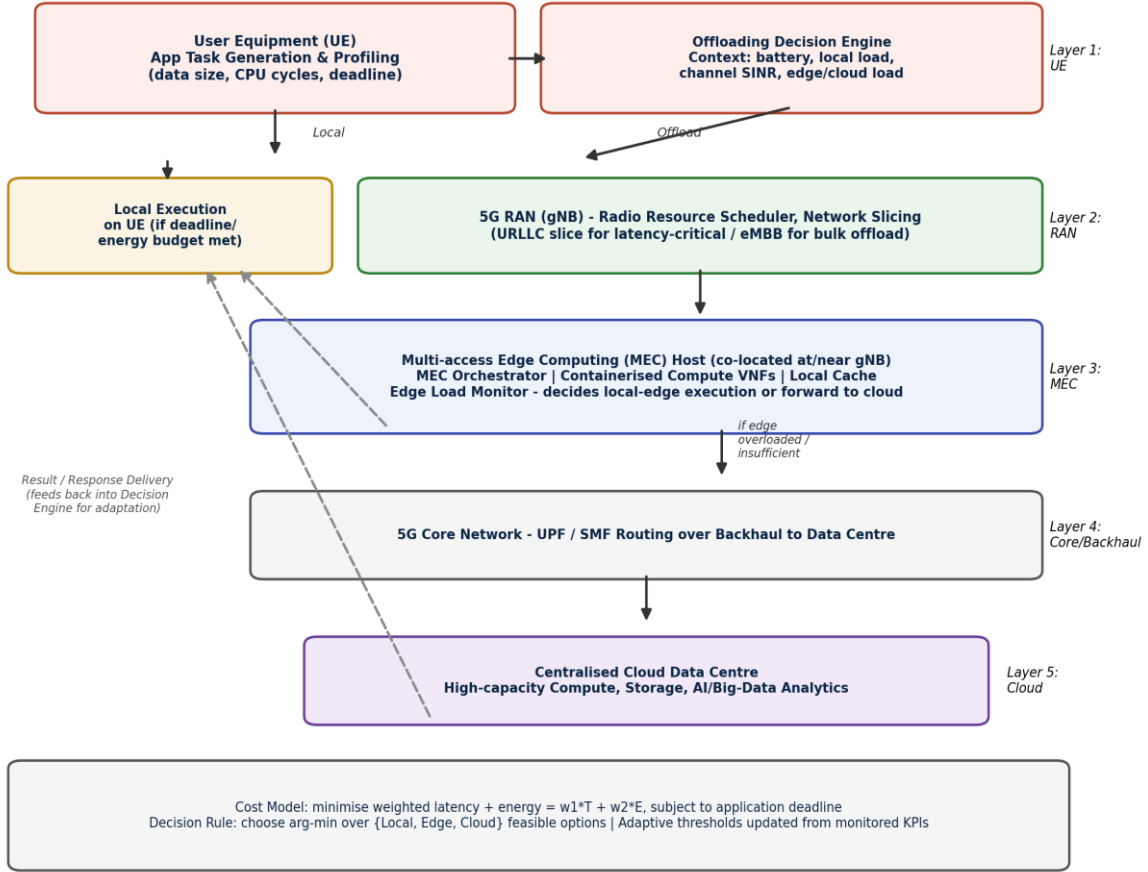


Figure 1: Task-offloading architecture and decision workflow between UE, edge and cloud in a 5G network

Layer	Component(s)	Primary Function
1. UE / Application	Task generation & profiling module, offloading decision engine, local execution runtime	Characterise each task (data size, CPU cycles, deadline), gather context (battery, local load, channel SINR), decide Local / Edge / Cloud
2. 5G RAN	gNB with radio resource scheduler and network slicing (URLLC for latency-critical, eMBB for bulk offload)	Admit and schedule uplink/downlink transmissions for offloaded tasks with appropriate QoS
3. MEC (Edge)	MEC host co-located at/near the gNB: orchestrator, containerised compute VNFs, local cache, load monitor	Execute offloaded tasks with minimal added latency; forward to cloud if locally overloaded or under-resourced
4. 5G Core / Backhaul	UPF, SMF and backhaul transport routing traffic between MEC, core and external data networks	Route offloaded traffic that cannot be served at the edge onward to the centralised cloud
5. Cloud	Centralised data-centre compute, storage and AI/big-data analytics platforms	Execute large-scale or non-latency-critical tasks with high aggregate compute capacity

2.1 Offloading Decision Model

Each task is characterised by its input data size D (bits), required CPU cycles C , and application deadline. The decision engine estimates, for each candidate location, the expected latency T and UE energy E , and evaluates a weighted cost function:

$$\text{Cost} = w_1 \cdot (T / \text{deadline}) + w_2 \cdot (E / E_{\text{max}}), \text{ subject to } T \leq \text{deadline}$$

where w_1 and w_2 are tunable weights reflecting the relative importance of latency versus energy for the application class. Any option that violates the deadline is treated as infeasible (infinite cost). The engine then selects the feasible option — Local, Edge or Cloud — with the minimum cost.

2.2 End-to-End Workflow

- The application layer on the UE generates a task and profiles its data size, CPU-cycle requirement and deadline.
- The offloading decision engine gathers context: UE battery level and local CPU load, current uplink SINR, and reported edge/cloud load.
- The engine evaluates the weighted cost of Local, Edge and Cloud execution and selects the minimum-cost feasible option.
- If Local is chosen, the task executes entirely on the UE and the result is used immediately.
- If Edge or Cloud is chosen, the task (and its input data) is transmitted over the 5G NR uplink — using a URLLC slice for latency-critical tasks or an eMBB slice for bulk data — to the gNB.
- The MEC host executes the task if it has sufficient capacity; if the edge is overloaded or the task exceeds edge capability, the MEC orchestrator forwards it via the 5G core (UPF/SMF) and backhaul to the cloud.
- The result is returned over the downlink to the UE, and the observed latency/energy outcome is fed back into the decision engine to refine future decisions.

3. Evaluation of Design

Table 2 evaluates the proposed architecture against its own design objectives, weighing the benefits of a network-aware, per-task decision engine against the operational gaps a deployment team must plan for.

Strengths	Weaknesses / Design Gaps
Per-task decision engine adapts to instantaneous channel and load conditions rather than a fixed policy	Decision engine needs accurate, low-overhead context signalling (SINR, edge load); stale context can misroute tasks
MEC co-location keeps latency low for compute-bound tasks — orders of magnitude faster than local execution for heavy tasks	MEC compute and storage capacity is finite per site; a popular application can exhaust edge resources under load spikes
Cloud fallback provides virtually unlimited compute for very large or non-urgent tasks	Backhaul RTT to the cloud (tens of ms) can violate tight deadlines for latency-critical applications if invoked incorrectly
Offloading substantially reduces UE energy consumption compared with local execution for compute-heavy tasks	Continuous uplink transmission for many concurrent devices increases RAN load and can raise interference in dense deployments
Network slicing (URLLC/eMBB) allows differentiated QoS for different offloading traffic classes	The weighted cost model requires application-specific tuning (weights w_1 , w_2) that may not generalise across all app types
Local-execution fallback preserves service continuity when radio coverage is poor	No explicit handling of user mobility — a task offloaded to one MEC host may need costly migration if the UE moves to another cell

The central design tension is that the edge tier offers the best latency for compute-bound tasks but has strictly limited capacity, while the cloud tier offers effectively unlimited capacity at the cost of extra backhaul delay;

the decision engine must continuously arbitrate between these two options (with local execution as a fallback) as conditions change, rather than committing to a single fixed policy.

4. Justification

The design choices above are justified quantitatively using standard latency and energy models for mobile edge computing (validated in the MATLAB analysis of Section 6).

4.1 Why Local Execution Alone Is Insufficient

For a representative compute-bound task, local latency grows linearly with task complexity and quickly exceeds a typical interactive-application deadline of 150 ms — for example, a 3 Mbit task requires roughly 2 seconds on a typical mobile CPU, more than ten times the deadline (Section 6, Figure 1). This justifies offloading as a necessary, not optional, part of the architecture for compute-heavy 5G applications.

4.2 Why Edge Beats Cloud for Most Tasks, and When Cloud Wins

Because the MEC host is co-located with the gNB, its propagation/scheduling overhead (about 1 ms) is far smaller than the cloud's backhaul round-trip (about 20 ms). For small-to-moderate tasks this makes edge execution the lowest-latency option even though the cloud has higher raw compute capacity. As task complexity grows, however, the cloud's much larger aggregate compute capacity eventually outweighs its extra backhaul delay, producing a crossover beyond which cloud offloading becomes optimal (observed at roughly 2.3–2.5 GCycles in the Section 6 analysis). This justifies including both edge and cloud as first-class options rather than routing every offloaded task to a single tier.

4.3 Why Offloading Saves Energy

The dynamic-power model (energy $\propto C \cdot f^2$) means local execution energy grows quickly with task complexity, while the UE's offloading energy is limited to the (much smaller) transmission energy for sending the task upstream. The MATLAB analysis shows local-execution energy exceeding transmission energy by roughly two to three orders of magnitude for moderate-to-large tasks, directly justifying offloading as the energy-preferred choice whenever the deadline and channel conditions allow it.

4.4 Why the Decision Must Depend on Channel Quality

The decision-region map (Section 6, Figure 4) shows that at very low uplink SINR, transmission time can grow large enough that local execution becomes competitive again for very small tasks, even though it is otherwise dominated by edge/cloud. This justifies making channel quality (not just task complexity) an explicit input to the decision engine, rather than using a static, channel-independent offloading policy.

5. Suggestions for Improvement

The following enhancements address the design gaps identified in Section 3 while preserving the core edge/cloud offloading architecture.

Improvement	Rationale / Expected Benefit
Predictive/learning-based decision engine (e.g., reinforcement learning)	Learns from observed latency/energy outcomes to refine offloading thresholds over time rather than relying on a static weighted-cost model
Service/task migration across MEC hosts	Proactively migrates an in-progress edge task to the target MEC host as the UE moves between cells, avoiding a costly re-offload

Improvement	Rationale / Expected Benefit
Partial (fine-grained) offloading	Splits a task into local and offloaded sub-components (e.g., DNN layer partitioning) instead of an all-or-nothing decision, improving resource utilisation
Multi-MEC load balancing / cooperative edge	Neighbouring MEC hosts share load information so an overloaded edge node can offload laterally to a nearby MEC before falling back to the distant cloud
Admission control & QoS-aware slicing	Reserves a URLLC slice specifically for deadline-critical offloaded tasks to prevent them being starved by bulk eMBB traffic
Energy-aware adaptive weighting (w1, w2)	Dynamically increases the energy weight as UE battery drops, biasing the decision engine toward offloading when battery is low
Security/isolation for offloaded execution	Container/VM-level isolation and encrypted task payloads at the MEC host protect multi-tenant edge infrastructure and user data

6. Supporting MATLAB Design Analysis

A MATLAB script (offload_strategy_analysis.m, provided in full below) quantitatively validates the design and justification arguments above: (1) latency comparison for Local/Edge/Cloud execution versus task size, (2) UE energy comparison, (3) the weighted latency-energy cost and resulting optimal decision versus task size, and (4) an optimal-decision region map over uplink SINR and task complexity. Key output plots are shown below; a representative latency table is given in Table 4.

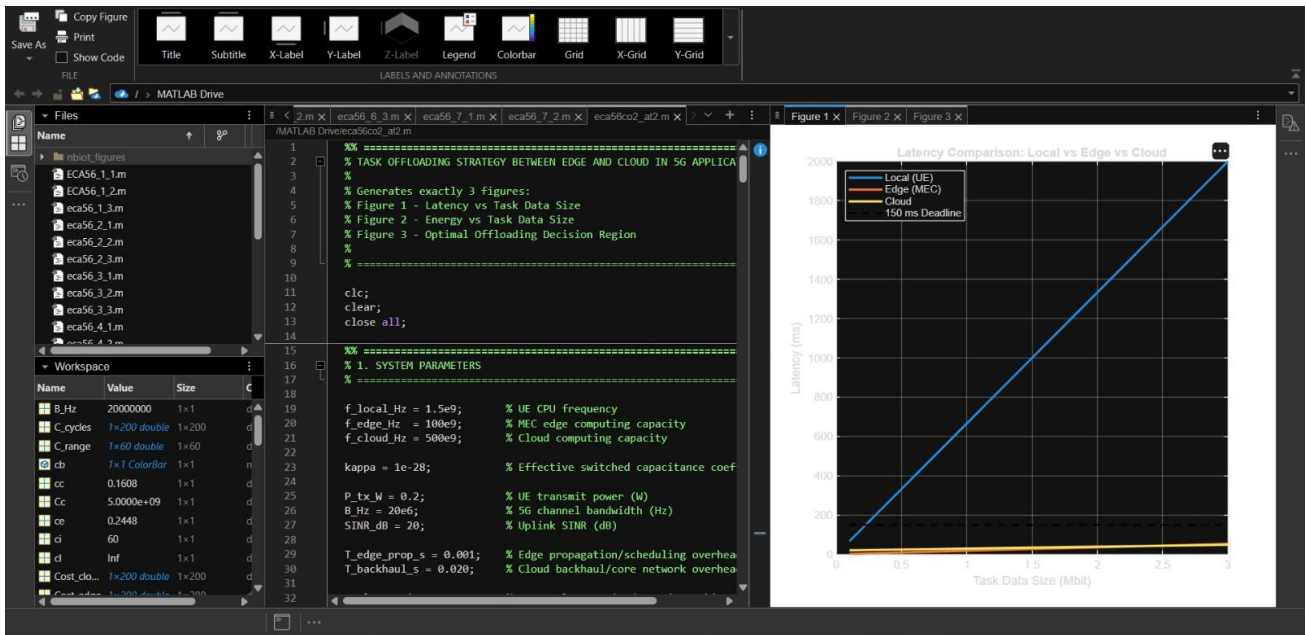


Figure 2: Latency comparison — Local vs Edge vs Cloud execution versus task data size

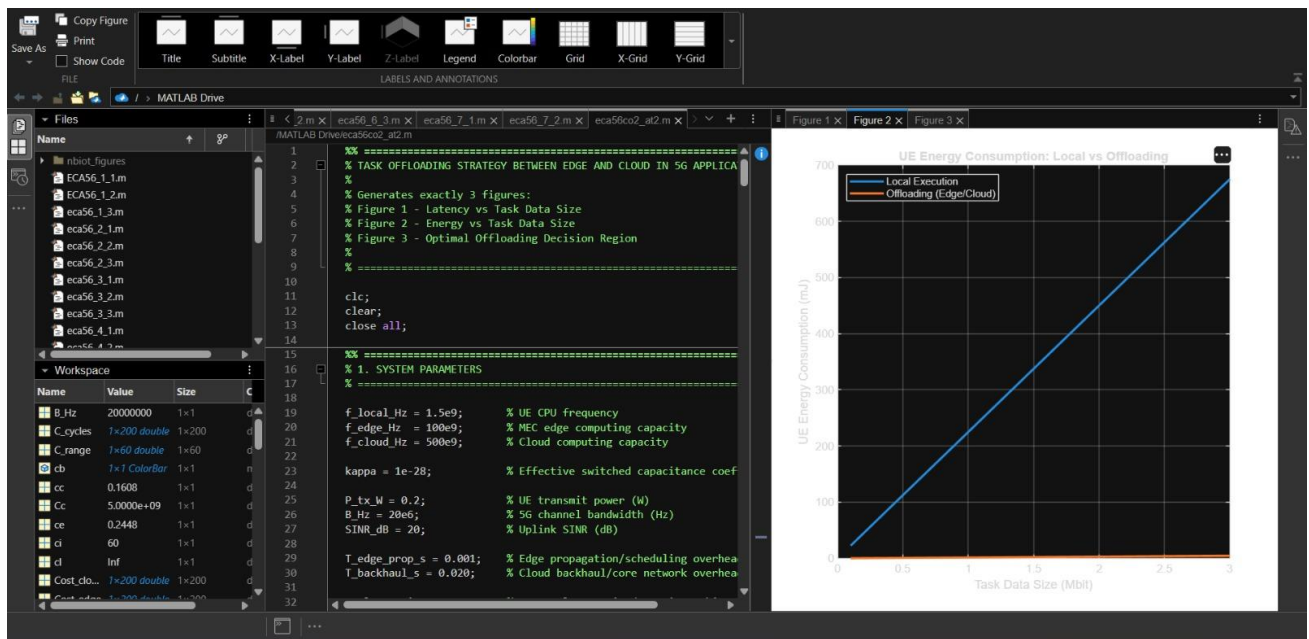


Figure 3: UE energy consumption — local execution vs offloading (transmission only)

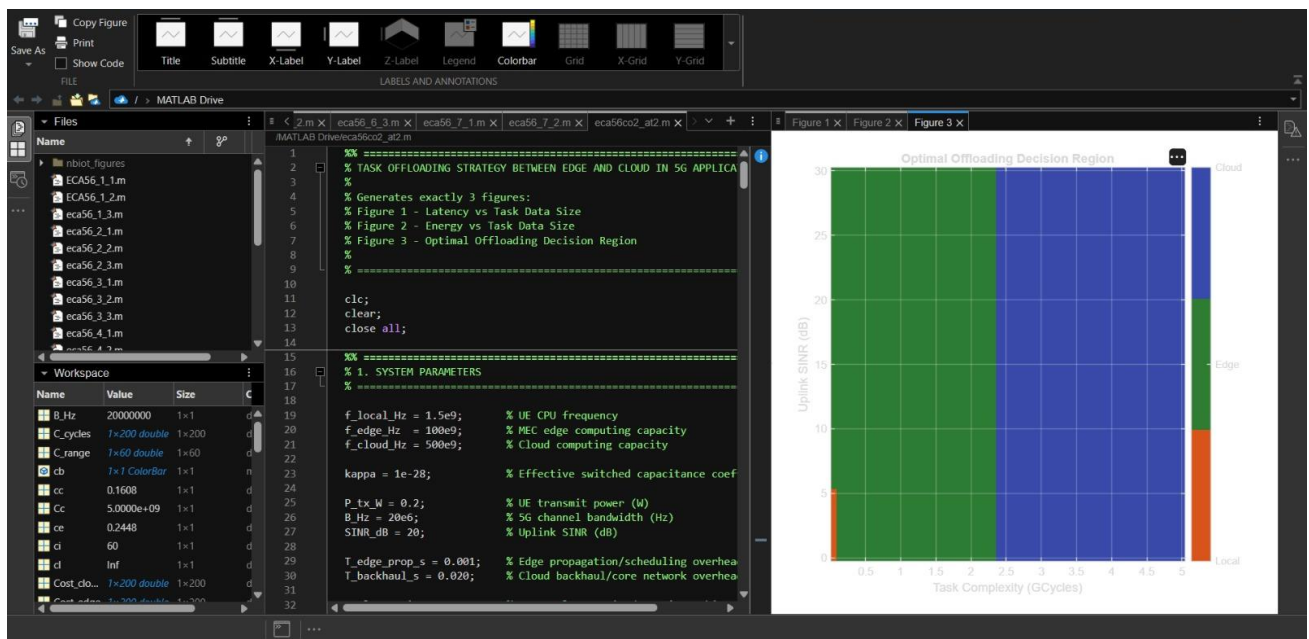


Figure 4: Optimal offloading decision region over uplink SINR and task complexity

Task Size (Mbit)	Local Latency (ms)	Edge Latency (ms)	Cloud Latency (ms)
0.10	66.7	2.8	21.0
0.83	552.4	15.5	27.9
1.56	1038.2	28.3	34.8
2.27	1514.2	40.8	41.6
3.00	2000.0	53.5	48.5

Appendix A: MATLAB Source Code (offload_strategy_analysis.m)

```
%% =====
% TASK OFFLOADING STRATEGY BETWEEN EDGE AND CLOUD IN 5G APPLICATIONS
% Supporting Design Analysis (CO3 - AT2, BL6, Question 6)
%
% This script quantitatively supports the offloading-strategy design:
% (1) Latency model: Local vs Edge vs Cloud execution vs task size
% (2) Energy model: Local vs Edge/Cloud (transmission) vs task size
% (3) Weighted cost model and optimal offloading decision boundary
% (4) Decision-region map over channel SINR vs task complexity
%
% Author : Design Task Submission - ECA56 (Advanced Wireless / 5G-6G)
% =====
clc; clear; close all;

%% -----
% SYSTEM PARAMETERS
% -----
f_local_Hz = 1.5e9;      % UE (mobile) CPU clock frequency (Hz)
f_edge_Hz  = 100e9;     % MEC edge server effective compute capacity (Hz, multi-core aggregate)
f_cloud_Hz = 500e9;     % Cloud data-centre effective compute capacity (Hz, multi-core aggregate)

kappa      = 1e-28;     % effective switched-capacitance coefficient (energy = kappa*C*f^2)
P_tx_W     = 0.2;       % UE transmit power (W) (~23 dBm)
B_Hz       = 20e6;      % allocated 5G NR channel bandwidth per UE (Hz)
SINR_dB    = 20;        % baseline uplink SINR (dB)

T_edge_prop_s = 0.001; % MEC co-location propagation/scheduling overhead (s)
T_backhaul_s  = 0.020; % additional core-network + backhaul RTT to cloud (s)

cyclesPerBit = 1000;    % computational complexity: CPU cycles required per bit of input data

Deadline_s = 0.150;     % example application latency deadline (150 ms, e.g. AR/interactive app)

w1 = 0.6; % weight on latency in the cost function
w2 = 0.4; % weight on energy in the cost function

%% -----
% SECTION 1: LATENCY vs TASK DATA SIZE
% -----
SINR_lin = 10^(SINR_dB/10);
R_bps = B_Hz*log2(1 + SINR_lin);          % Shannon achievable uplink data rate

D_bits = linspace(0.1e6, 3e6, 200);      % task input data size: 0.1 - 3 Mbit
C_cycles = D_bits*cyclesPerBit;           % required CPU cycles for the task

T_local = C_cycles/f_local_Hz;
T_tx     = D_bits/R_bps;
T_edge   = T_tx + C_cycles/f_edge_Hz + T_edge_prop_s;
T_cloud  = T_tx + C_cycles/f_cloud_Hz + T_backhaul_s;

figure('Name','Latency vs Data Size','Color','w');
plot(D_bits/1e6, T_local*1000, 'LineWidth',1.8); hold on;
plot(D_bits/1e6, T_edge*1000, 'LineWidth',1.8);
plot(D_bits/1e6, T_cloud*1000, 'LineWidth',1.8);
if exist('yline','builtin') || exist('yline','file')
    yline(Deadline_s*1000, '--k', 'Deadline');
else
    plot(D_bits/1e6, Deadline_s*1000*ones(size(D_bits)), '--k', 'LineWidth', 1.2);
end
grid on; xlabel('Task Data Size (Mbit)'); ylabel('Latency (ms)');
legend('Local (UE)','Edge (MEC)','Cloud','Location','northwest');
title('Latency Comparison: Local vs Edge vs Cloud Execution');
saveas(gcf, 'fig1_latency_vs_datasize.png');

fprintf('=====\n');
fprintf('SECTION 1: LATENCY AT SELECTED TASK SIZES (SINR = %d dB)\n', SINR_dB);
fprintf('=====\n');
sampleIdx = round(linspace(1, numel(D_bits), 5));
for i = sampleIdx
    fprintf('D = %5.2f Mbit | T_local = %7.2f ms | T_edge = %6.2f ms | T_cloud = %6.2f ms\n', ...
        D_bits(i)/1e6, T_local(i)*1000, T_edge(i)*1000, T_cloud(i)*1000);
end
```

```

%% -----
% SECTION 2: ENERGY vs TASK DATA SIZE
% -----
E_local = kappa*C_cycles.*(f_local_Hz^2); % Joules, UE-side dynamic energy
E_offload = P_tx_W*T_tx; % Joules, UE-side transmit energy (same for edge/cloud)

figure('Name','Energy vs Data Size','Color','w');
plot(D_bits/1e6, E_local*1000, 'LineWidth',1.8); hold on;
plot(D_bits/1e6, E_offload*1000, 'LineWidth',1.8);
grid on; xlabel('Task Data Size (Mbit)'); ylabel('UE Energy Consumption (mJ)');
legend('Local Execution','Offload (Edge or Cloud) - Tx only','Location','northwest');
title('UE Energy Consumption: Local Execution vs Offloading');
saveas(gcf, 'fig2_energy_vs_datasize.png');

%% -----
% SECTION 3: WEIGHTED COST AND OPTIMAL DECISION vs TASK SIZE
% -----
% Normalise latency and energy before combining (min-max over the sweep)
normT_local = T_local/max(T_cloud); normE_local = E_local/max(E_local);
normT_edge = T_edge /max(T_cloud); normE_edge = E_offload/max(E_local);
normT_cloud = T_cloud/max(T_cloud); normE_cloud = E_offload/max(E_local);

Cost_local = w1*normT_local + w2*normE_local;
Cost_edge = w1*normT_edge + w2*normE_edge;
Cost_cloud = w1*normT_cloud + w2*normE_cloud;

% infeasible options (exceed deadline) are penalised heavily
Cost_local(T_local > Deadline_s) = Inf;
Cost_edge(T_edge > Deadline_s) = Inf;
Cost_cloud(T_cloud > Deadline_s) = Inf;

[~, decision] = min([Cost_local; Cost_edge; Cost_cloud], [], 1);
% decision: 1 = Local, 2 = Edge, 3 = Cloud

figure('Name','Weighted Cost vs Data Size','Color','w');
plot(D_bits/1e6, Cost_local, 'LineWidth',1.8); hold on;
plot(D_bits/1e6, Cost_edge, 'LineWidth',1.8);
plot(D_bits/1e6, Cost_cloud, 'LineWidth',1.8);
grid on; xlabel('Task Data Size (Mbit)'); ylabel('Weighted Cost (normalised)');
legend('Local','Edge','Cloud','Location','northwest');
title('Weighted Latency-Energy Cost vs Task Size');
saveas(gcf, 'fig3_cost_vs_datasize.png');

fprintf('\n=====\\n');
fprintf(' SECTION 3: OPTIMAL OFFLOADING DECISION (SAMPLE POINTS)\\n');
fprintf('=====\\n');
labels = {'Local','Edge','Cloud'};
for i = sampleIdx
    fprintf('D = %5.2f Mbit -> Optimal decision: %s\\n', D_bits(i)/1e6, labels{decision(i)});
end

%% -----
% SECTION 4: DECISION-REGION MAP (SINR vs TASK COMPLEXITY)
% -----
SINR_range_dB = linspace(0, 30, 60);
C_range = linspace(0.05e9, 5e9, 60); % CPU-cycle requirement sweep

DecisionMap = zeros(numel(SINR_range_dB), numel(C_range));
D_fixed_bits = 2e6; % fixed representative data size for this map (2 Mbit)

for si = 1:numel(SINR_range_dB)
    sinr_lin = 10^(SINR_range_dB(si)/10);
    Rb = B_Hz*log2(1 + sinr_lin);
    Ttx = D_fixed_bits/Rb;
    for ci = 1:numel(C_range)
        Cc = C_range(ci);
        Tl = Cc/f_local_Hz;
        Te = Ttx + Cc/f_edge_Hz + T_edge_prop_s;
        Tc = Ttx + Cc/f_cloud_Hz + T_backhaul_s;

        El = kappa*Cc*(f_local_Hz^2);
        Eo = P_tx_W*Ttx;

        cl = w1*(Tl/Deadline_s) + w2*(El/max(El,1e-9));
    end
end

```

```

ce = w1*(Te/Deadline_s) + w2*(Eo/max(E1,1e-9));
cc = w1*(Tc/Deadline_s) + w2*(Eo/max(E1,1e-9));

if Tl > Deadline_s, cl = Inf; end
if Te > Deadline_s, ce = Inf; end
if Tc > Deadline_s, cc = Inf; end

[~, idx] = min([cl, ce, cc]);
DecisionMap(si, ci) = idx;
end
end

figure('Name','Decision Region Map','Color','w');
imagesc(C_range/1e9, SINR_range_dB, DecisionMap);
set(gca,'YDir','normal');
colormap([0.85 0.33 0.10; 0.18 0.49 0.20; 0.22 0.29 0.67]); % Local=orange, Edge=green, Cloud=blue
try
    cb = colorbar; cb.Ticks = [1 2 3]; cb.TickLabels = {'Local','Edge','Cloud'};
catch
    colorbar; % fallback for environments where colorbar tick labelling is unavailable
end
xlabel('Task Complexity (GCycles)'); ylabel('Uplink SINR (dB)');
title('Optimal Offloading Decision Region (D = 2 Mbit fixed)');
saveas(gcf, 'fig4_decision_region_map.png');

fprintf('\nAnalysis complete. Figures generated for report embedding.\n');

```